PW SKILLS

Introduction to databases

Introduction to SQL Basics



SQL (Structured Query Language)

Standard language used to manage and manipulate data in relational databases. Relational databases store data in structured formats, typically using tables with rows and columns. SQL allows you to interact with these databases by performing operations such as retrieving, inserting, updating, and deleting records.

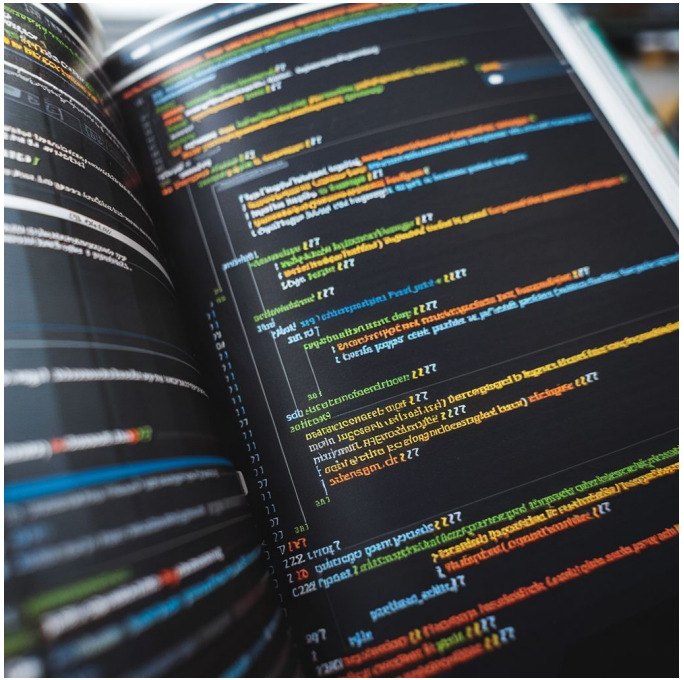
SQL is widely used with databases like MySQL, PostgreSQL, SQLite, and Oracle.



Purpose of SQL

SQL is used for:

- Querying data
- Managing data
- Defining data
- Controlling access



SQL Syntax

SQL queries have a specific syntax and are usually made up of statements like:

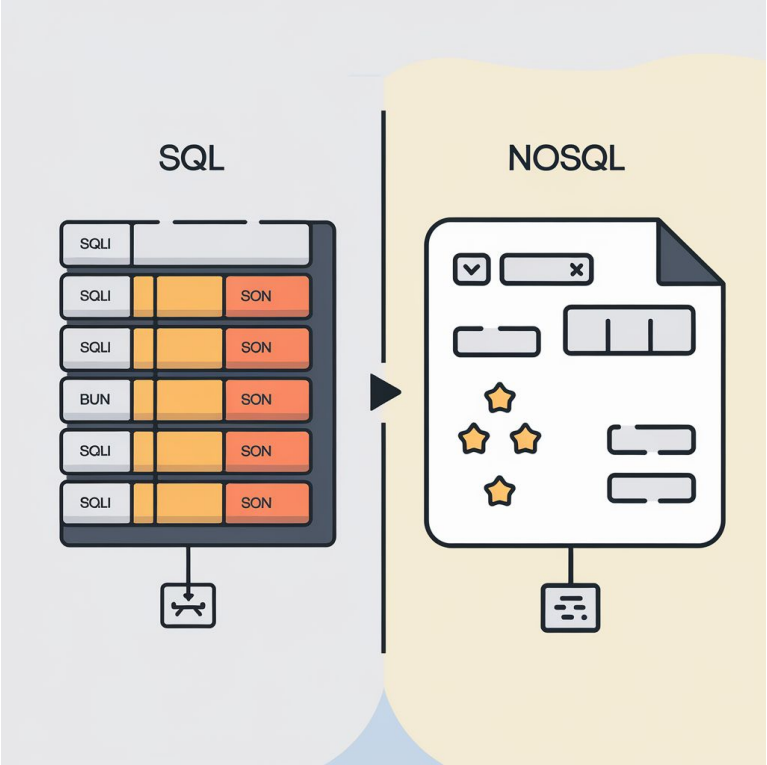
- **SELECT:** To fetch data from the database.
- **INSERT:** To add data to a table.
- **UPDATE:** To modify existing data.
- **DELETE:** To remove data from a table.

SQL vs NoSQL

Aspect	SQL (Relational Databases)	NoSQL (Non-relational Databases)
Structure	Structure(tables, rows, columns)	Flexible (JSON, key-value pairs, documents, etc.)
Data Schema	Predefined schema (fixed) Schema-less or	Dynamic schema
Scalability	Vertical (scaling up)	Horizontal (scaling out)
Examples	MySQL, PostgreSQL, SQLite, Oracle	MongoDB, Cassandra, Couchbase, Firebase
Use Case	Well-suited for transactional applications	Ideal for handling large volumes of unstructured or semi-structured data

SQL Databases: They are structured and relational, with predefined schemas and tables. They use **Structured Query Language (SQL)** to interact with the database. They are best for applications that need strong data consistency and integrity.

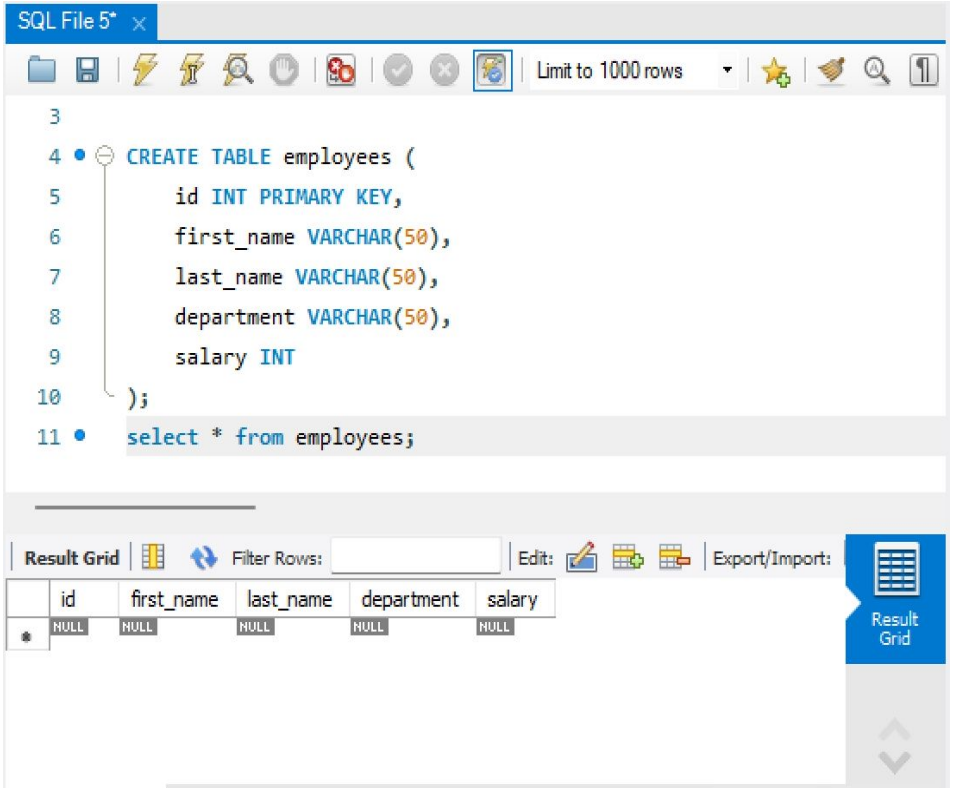
NoSQL Databases: They are non-relational and allow flexibility in how data is stored, often used for applications requiring large-scale data management, rapid development, and flexibility in data format. They are best suited for handling **unstructured** or **semi-structured data.**

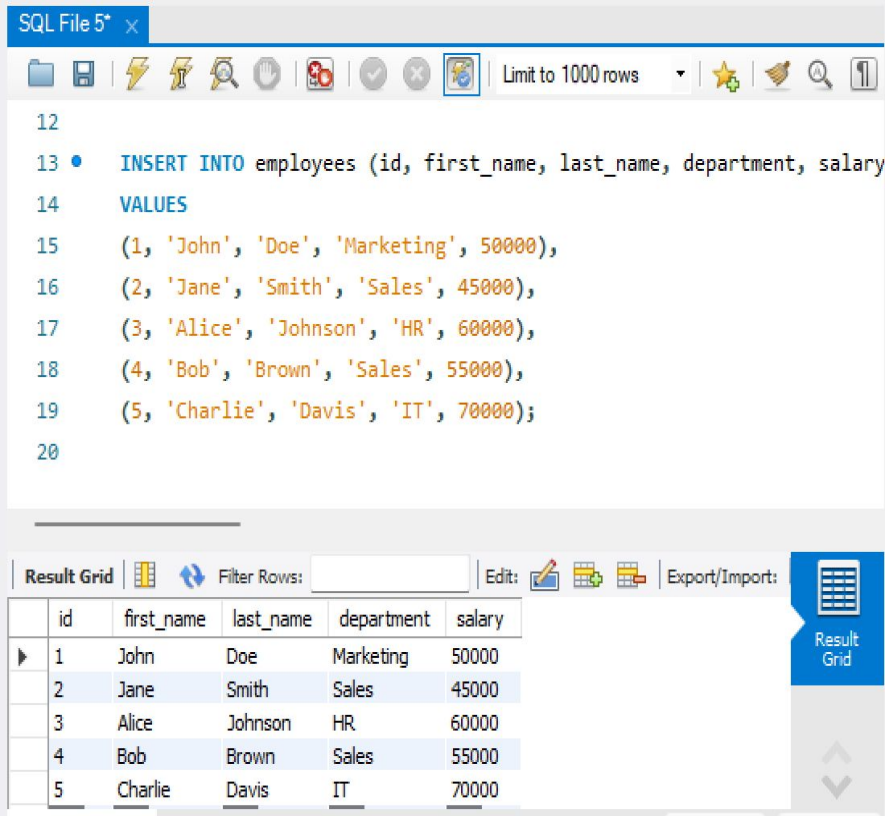


Example Dataset

Let's assume we are working with a company's employee database. We can create a table `employees` with the following columns:

- **id** (INTEGER, Primary Key)
- **first_name** (VARCHAR)
- **last_name** (VARCHAR)
- **department** (VARCHAR)
- **salary** (INTEGER)





Query Examples:

- **Select all employees:**

```
SELECT * FROM employees;
```

- **Select employees from the Sales department:**

```
SELECT first_name, last_name FROM
employees WHERE department = 'Sales';
```

- **Update an employee's salary:**

```
UPDATE employees SET salary = 60000
WHERE id = 2;
```

- **Delete an employee record:**

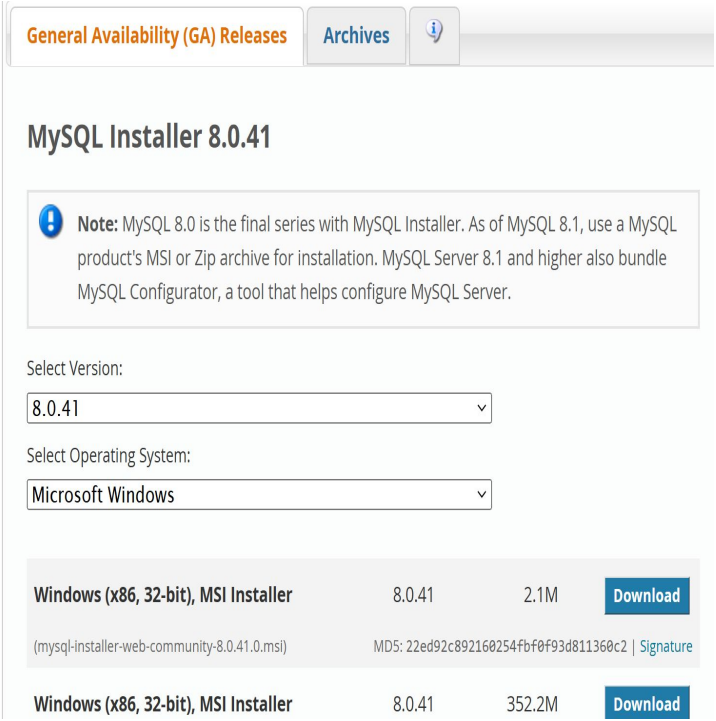
```
DELETE FROM employees WHERE id = 1;
```

Set Up a Database (MySQL/PostgreSQL)

Step 1: Install MySQL or PostgreSQL

For MySQL:

1. **Download MySQL:**
    1. Go to the official ***MySQL download page.***
    2. Choose the version that suits your operating system (Windows, macOS, Linux).
    3. Run the installer and follow the on-screen instructions to complete the installation.



Step 2: Create a Database

In MySQL:

Create a database:

```
CREATE DATABASE database_name;
```

1. Replace database_name with the name you want for your database.

Create a new database

```
cursor.execute("CREATE DATABASE IF NOT EXISTS my_database")
```

Create a new Table

```
cursor.execute("""
    CREATE TABLE IF NOT EXISTS users (
        id INT AUTO_INCREMENT PRIMARY KEY,
        name VARCHAR(100) NOT NULL,
        email VARCHAR(100) UNIQUE NOT NULL,
        age INT
    )
""")

print("Table created successfully!")
cursor.close()
conn.close()
```

Step 3: Connecting to a Database

In MySQL:

To connect to your database:

```
mysql -u username -p
```

You'll be prompted for your password. After logging in, select the database you want to work with:

```
USE database_name;
```

```
import mysql.connector

conn = mysql.connector.connect(
    host="your_host",
    user="your_username",
    password="your_password",
    database="your_database"
)

cursor = conn.cursor()
cursor.execute("SELECT DATABASE();")
print(cursor.fetchone())

cursor.close()
conn.close()
```

Basic Data Retrieval with SELECT in SQL

1. **SELECT All Columns**

To retrieve all columns from a table, you can use the * symbol, which stands for "all columns."

Example:

```
SELECT * FROM employees;
```

This query retrieves all the data from the `employees` table, including all columns for every row.

```
22 •  SELECT * FROM employees;
23
```

Result Grid | Filter Rows: | Edit:

id	first_name	last_name	department	salary
1	John	Doe	Marketing	50000
2	Jane	Smith	Sales	45000
3	Alice	Johnson	HR	60000
4	Bob	Brown	Sales	55000
5	Charlie	Davis	IT	70000

Basic Data Retrieval with SELECT in SQL

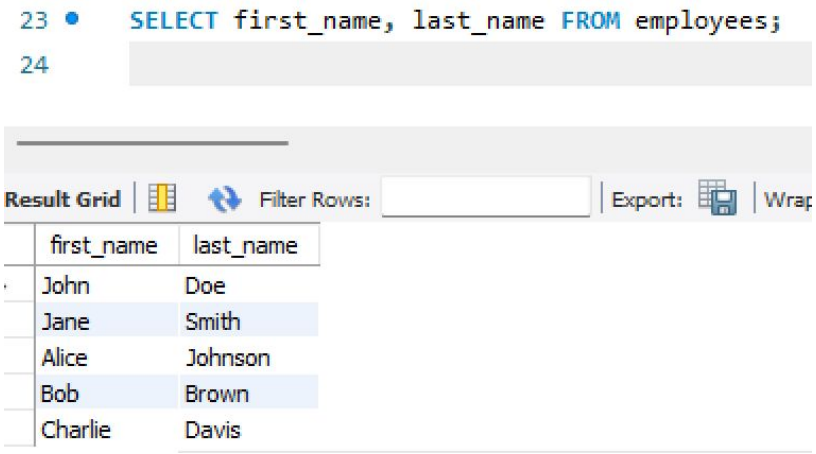
SELECT Specific Columns

To retrieve only specific columns from a table, list them in the `SELECT` statement, separated by commas.

Example:

```
SELECT first_name, last_name FROM employees;
```

This query retrieves only the `first_name` and `last_name` columns from the `employees` table.



Basic Data Retrieval with SELECT in SQL

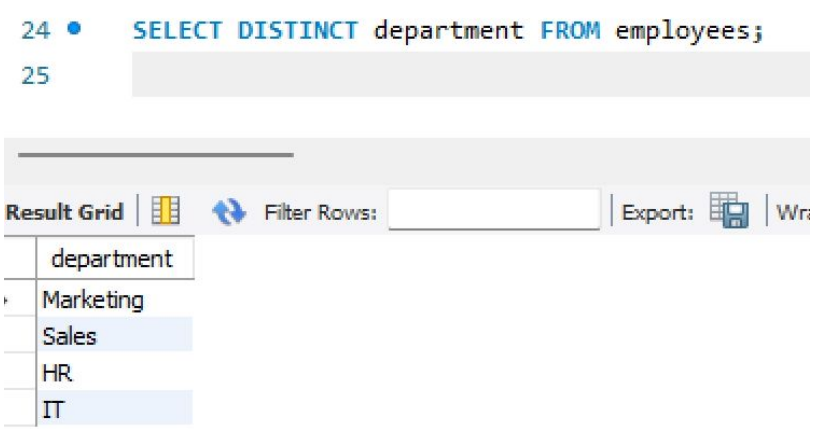
Using DISTINCT to Avoid Duplicates

If you want to get unique (non-duplicate) values in a column, use the `DISTINCT` keyword. It ensures that the result set contains only unique values for the specified column.

Example:

```
SELECT DISTINCT department FROM employees;
```

This query retrieves a list of unique departments from the `employees` table, removing any duplicate department values.



Quiz

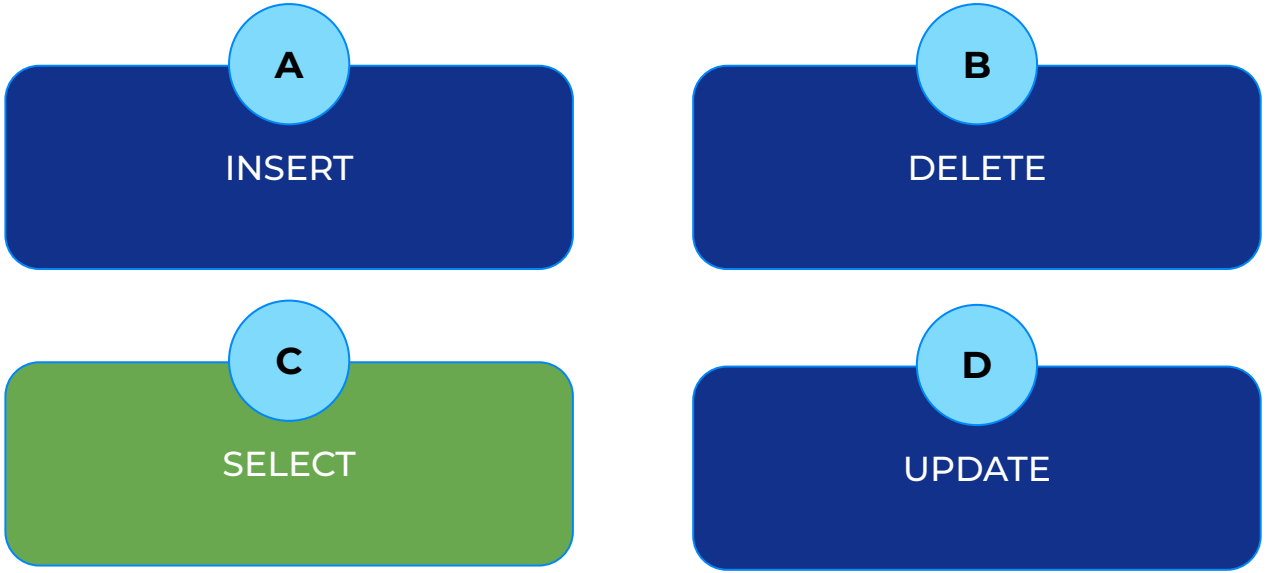
Question: Which SQL command is used to retrieve data from a table?

A. INSERT

B. DELETE

C. SELECT

D. UPDATE



Explanation:
The SELECT statement retrieves data from one or more tables. For example:

```
SELECT * FROM employees;
```

- INSERT adds data, DELETE removes data, and UPDATE modifies data.

Using WHERE Clause for Filtering Data

The `WHERE` clause is used to filter data based on specific conditions. It can be used with comparison operators (like `=, >, <, >=, <=, !=`), logical operators (like `AND, OR`), and pattern-matching operators (`LIKE, BETWEEN, IN`).

Basic Comparison Operators:

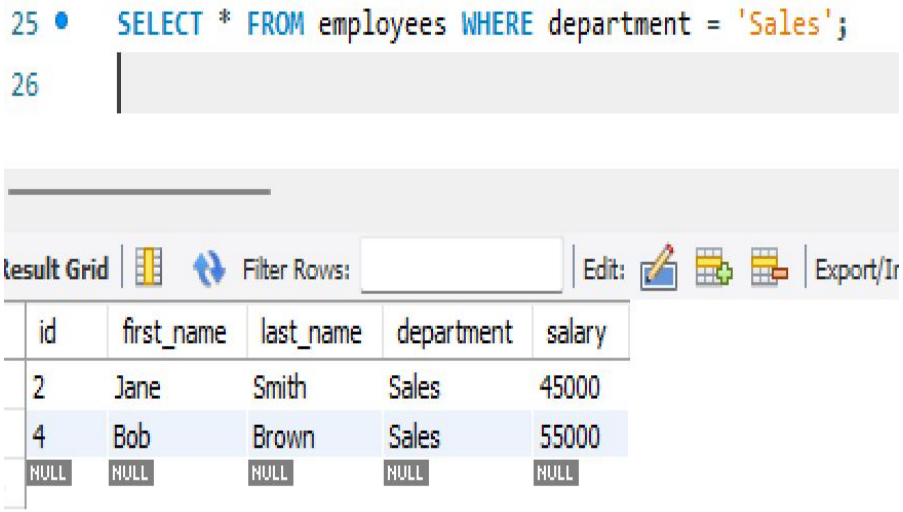
- `=`: Equal to
- `>`: Greater than
- `<`: Less than
- `>=`: Greater than or equal to
- `<=`: Less than or equal to
- `!=`: Not equal to

Using WHERE Clause for Filtering Data

Example 1: Filtering with a Single Condition

```
SELECT * FROM employees WHERE department = 'Sales';
```

This query retrieves all rows from the `employees` table where the `department` is 'Sales'.



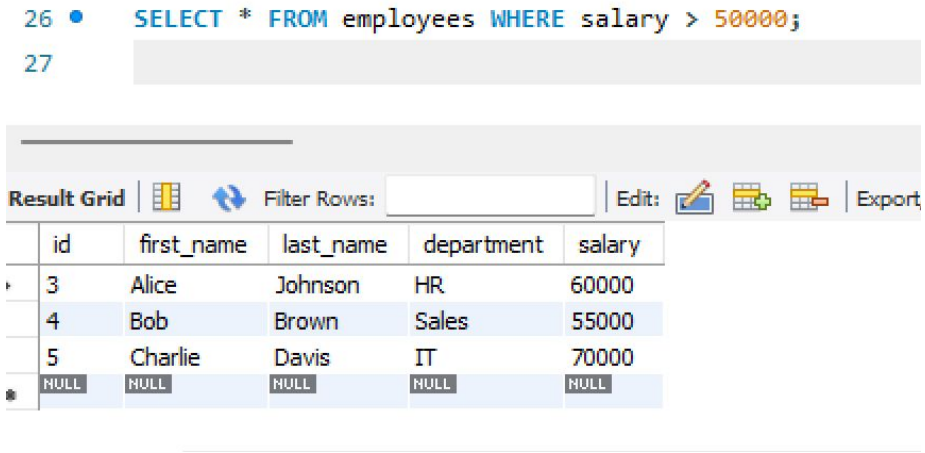
id	first_name	last_name	department	salary
2	Jane	Smith	Sales	45000
4	Bob	Brown	Sales	55000
NULL	NULL	NULL	NULL	NULL

Using WHERE Clause for Filtering Data

Example 2: Filtering with Comparison Operators

```
SELECT * FROM employees WHERE salary > 50000;
```

This query retrieves all employees whose salary is greater than 50,000.

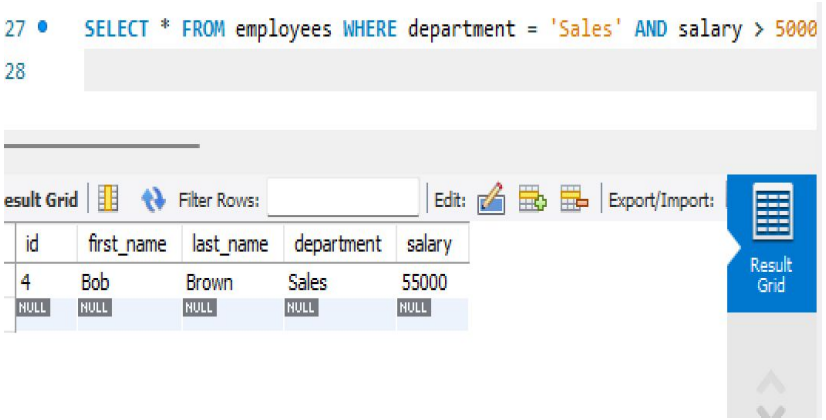


Using WHERE Clause for Filtering Data

Example 3: Using Multiple Conditions with AND

```
SELECT * FROM employees WHERE department =
'Sales' AND salary > 50000;
```

This query retrieves employees from the `Sales` department whose salary is greater than 50,000.

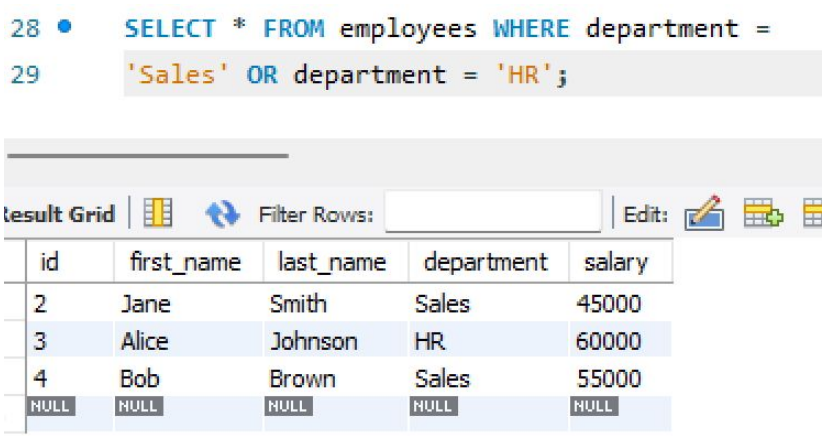


Using WHERE Clause for Filtering Data

Example 4: Using Multiple Conditions with `OR`

```
SELECT * FROM employees WHERE department =
'Sales' OR department = 'HR';
```

This query retrieves employees from either the `Sales` or `HR` department.



Using WHERE Clause for Filtering Data

Example 5: Using `BETWEEN` for Range Filtering

```
SELECT * FROM employees WHERE salary
BETWEEN 40000 AND 60000;
```

This query retrieves employees whose salary is between 40,000 and 60,000 (inclusive).

```
1 •  SELECT * FROM employees WHERE
2    salary BETWEEN 40000 AND 60000;
```

sult Grid | Filter Rows: | Edit:

id	first_name	last_name	department	salary
1	John	Doe	Marketing	50000
2	Jane	Smith	Sales	45000
3	Alice	Johnson	HR	60000
4	Bob	Brown	Sales	55000
NULL	NULL	NULL	NULL	NULL

Using WHERE Clause for Filtering Data

Example 6: Using `IN` to Filter Multiple Values

```
SELECT * FROM employees WHERE department IN
('Sales', 'HR', 'IT');
```

This query retrieves employees who are in the `Sales`, `HR`, or `IT` departments.

```
33
34 •  SELECT * FROM employees WHERE
35    department IN ('Sales', 'HR', 'IT');
```

Result Grid | Filter Rows: | Edit:

id	first_name	last_name	department	salary
2	Jane	Smith	Sales	45000
3	Alice	Johnson	HR	60000
4	Bob	Brown	Sales	55000
5	Charlie	Davis	IT	70000
NULL	NULL	NULL	NULL	NULL

Key Points:

- The `SELECT *` retrieves all columns, while specifying column names allows you to limit the data.
- The `DISTINCT` keyword helps eliminate duplicate values.
- The `WHERE` clause is used to filter the result set based on conditions like comparison operators, logical operators, and more.
- You can use logical operators (`AND, OR`) to combine conditions.
- Use `BETWEEN` and `IN` for range and multi-value filtering.

Quiz

Question: What does the WHERE clause do in SQL?

A

Sorts the results

B

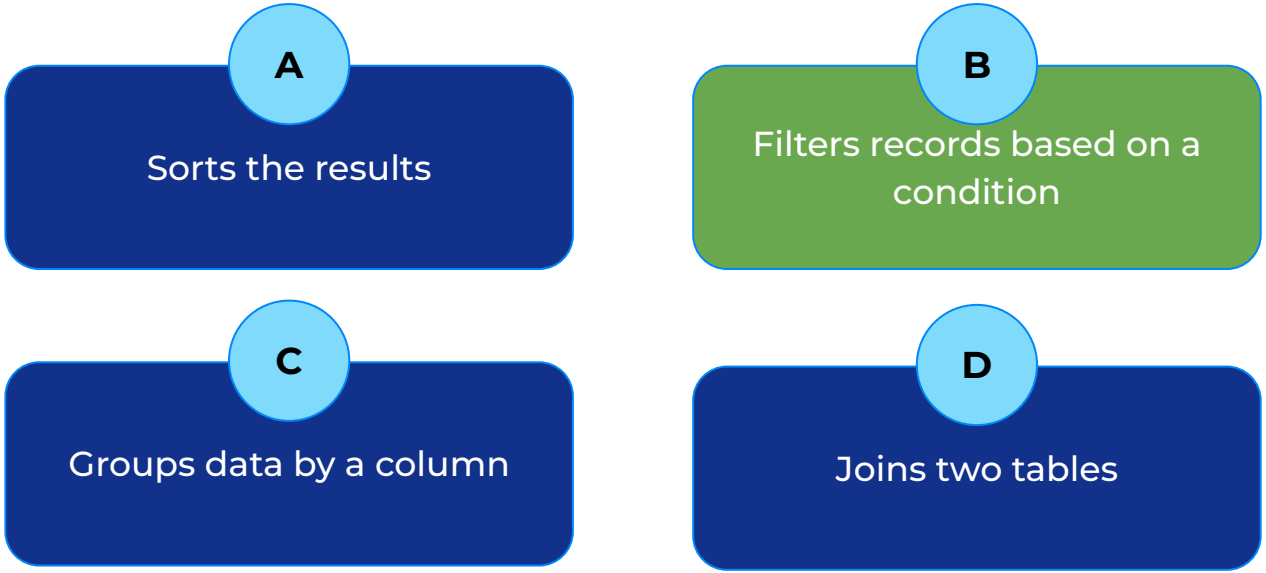
Filters records based on a condition

C

Groups data by a column

D

Joins two tables



The WHERE clause specifies conditions to filter rows. Example

```
SELECT * FROM employees WHERE department = 'Sales';
```

- Sorting uses ORDER BY, grouping uses GROUP BY, and joining uses JOIN.

Manipulating Data: INSERT, UPDATE, DELETE

SQL provides commands to manipulate data within tables. These commands allow you to add new records (**INSERT**), modify existing records (**UPDATE**), and remove records (**DELETE**).

1. INSERT: Adding Data to a Table

The `INSERT` statement is used to add new rows to a table. You can specify the columns into which data should be inserted.

Basic Syntax:

```
INSERT INTO table_name (column1, column2, column3, ...)

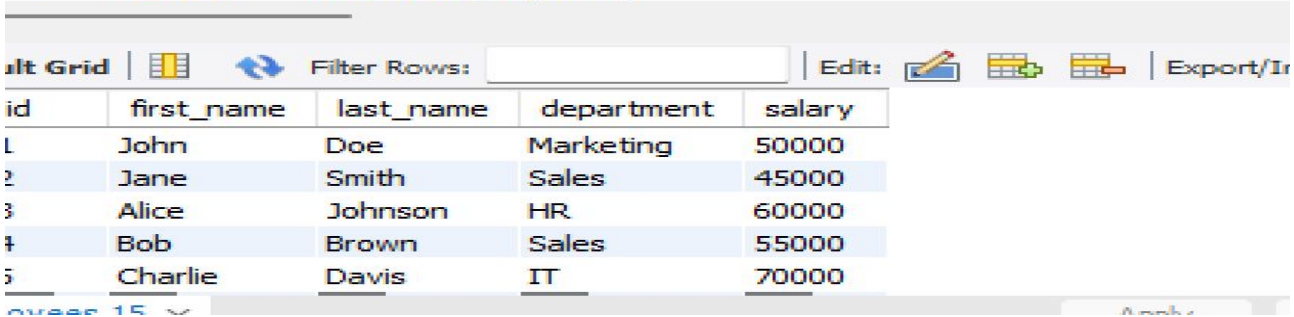
VALUES (value1, value2, value3, ...);
```

Example:

```
INSERT INTO employees (id, first_name, last_name, depar
VALUES
(1, 'John', 'Doe', 'Marketing', 50000),
(2, 'Jane', 'Smith', 'Sales', 45000),
(3, 'Alice', 'Johnson', 'HR', 60000),
(4, 'Bob', 'Brown', 'Sales', 55000),
(5, 'Charlie', 'Davis', 'IT', 70000);
```

```
SELECT first_name, last_name FROM employees WHERE depar
```

```
SELECT * FROM employees;
```



id	first_name	last_name	department	salary
1	John	Doe	Marketing	50000
2	Jane	Smith	Sales	45000
3	Alice	Johnson	HR	60000
4	Bob	Brown	Sales	55000
5	Charlie	Davis	IT	70000

Inserting Multiple Rows:

You can also insert multiple rows in a single query.

```
INSERT INTO employees (first_name,
last_name, department, salary)
VALUES
('Emma', 'Johnson', 'Sales', 60000),

('Liam', 'Smith', 'IT', 70000),

('Olivia', 'Brown', 'HR', 45000);
```

```
50 •  INSERT INTO employees2 (first_name, last_name, department, salary)
51    VALUES
52    ('Emma', 'Johnson', 'Sales', 60000),
53    ('Liam', 'Smith', 'IT', 70000),
54    ('Olivia', 'Brown', 'HR', 45000);
55 •  select * from employees2;
56
```

Result Grid | Filter Rows: | Edit: | Export/Import:

id	first_name	last_name	department	salary
1	Emma	Johnson	Sales	60000
2	Liam	Smith	IT	70000
3	Olivia	Brown	HR	45000
NULL	NULL	NULL	NULL	NULL

2. UPDATE: Modifying Data

The `UPDATE` statement is used to modify existing records in a table.

Basic Syntax:

```
UPDATE table_name

SET column1 = value1, column2 = value2, ...

WHERE condition;
```

- **Note:** Always use the **WHERE** clause to avoid updating all rows in the table. Without it, the update will apply to every row.

Example:

```
UPDATE employees
SET salary = 65000
WHERE first_name = 'David' AND last_name =
'Williams';
```

This query updates the salary of the employee with the first name `David` and last name `Williams.`

3. DELETE: Removing Data

The DELETE statement is used to remove records from a table. You can delete all rows or use a WHERE clause to delete specific rows.

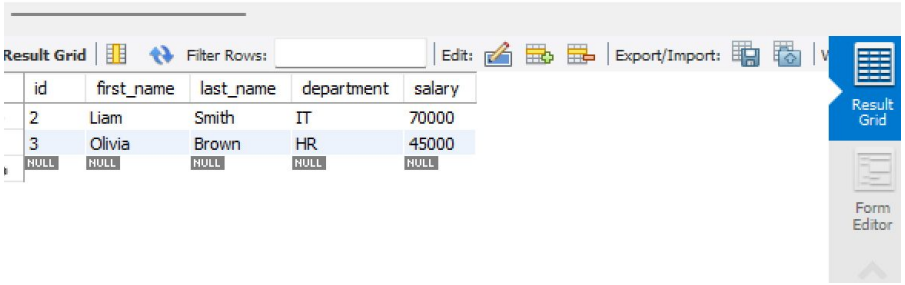
Basic Syntax:

DELETE FROM table_name WHERE condition;

Example:

DELETE FROM employees WHERE first_name = 'Olivia' AND last_name = 'Brown';

```
1  use  sql_class_1;
2  SELECT * FROM sql_class_1.employees2;
3
4  DELETE FROM employees2 WHERE first_name = 'Emma' AND last_name = 'Johnson';
```



id	first_name	last_name	department	salary
2	Liam	Smith	IT	70000
3	Olivia	Brown	HR	45000
NULL	NULL	NULL	NULL	NULL

Applying Data Integrity with Constraints

Constraints are rules enforced on data columns in SQL tables to maintain data accuracy and integrity. Common constraints include `NOT NULL`, `UNIQUE`, `DEFAULT`, and `CHECK`.

1. NOT NULL Constraint

The `NOT NULL` constraint ensures that a column cannot have a `NULL` value. This is useful when a column must always have a value.

Example:

```
CREATE TABLE employees (
    id INT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    department VARCHAR(50),
    salary INT
);
```

2. UNIQUE Constraint

The `UNIQUE` constraint ensures that all values in a column are distinct. No two rows can have the same value in a column defined with the `UNIQUE` constraint.

Example:

```
CREATE TABLE employees (

    id INT PRIMARY KEY,

    email VARCHAR(100) UNIQUE

);
```

In this example, the `email` column is set as `UNIQUE`, ensuring that no two employees can have the same email address.

3. DEFAULT Constraint

The `DEFAULT` constraint provides a default value for a column if no value is specified during an insert.

Example:

```
CREATE TABLE employees (

    id INT PRIMARY KEY,

    first_name VARCHAR(50),

    last_name VARCHAR(50),

    department VARCHAR(50) DEFAULT 'Sales',

    salary INT DEFAULT 50000

);
```

In this example, if no department or salary is provided when inserting a new row, the default values of 'Sales' and 50000 will be used.

4. CHECK Constraint

The `CHECK` constraint is used to enforce custom validation rules for data entry, ensuring that the data inserted into a column meets specific conditions.

Example:

```
CREATE TABLE employees (
    id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    salary INT,
    CHECK (salary >= 30000)
);
```

In this example, the `CHECK` constraint ensures that the salary value must be at least 30,000.

Summary

- **INSERT:** Adds new rows to a table. Use RETURNING in PostgreSQL and LAST_INSERT_ID() in MySQL to retrieve values after insertion.
- **UPDATE:** Modifies existing data. You can use the RETURNING clause in PostgreSQL to get updated values.
- **DELETE:** Removes rows from a table. Use a WHERE clause to specify which rows to delete.
- **Data Integrity Constraints:**
  - `NOT NULL:` Ensures columns cannot have NULL values.
  - `UNIQUE:` Ensures all values in a column are distinct.
  - `DEFAULT:` Provides default values for columns.
  - `CHECK:` Enforces custom validation rules for data.

Quiz

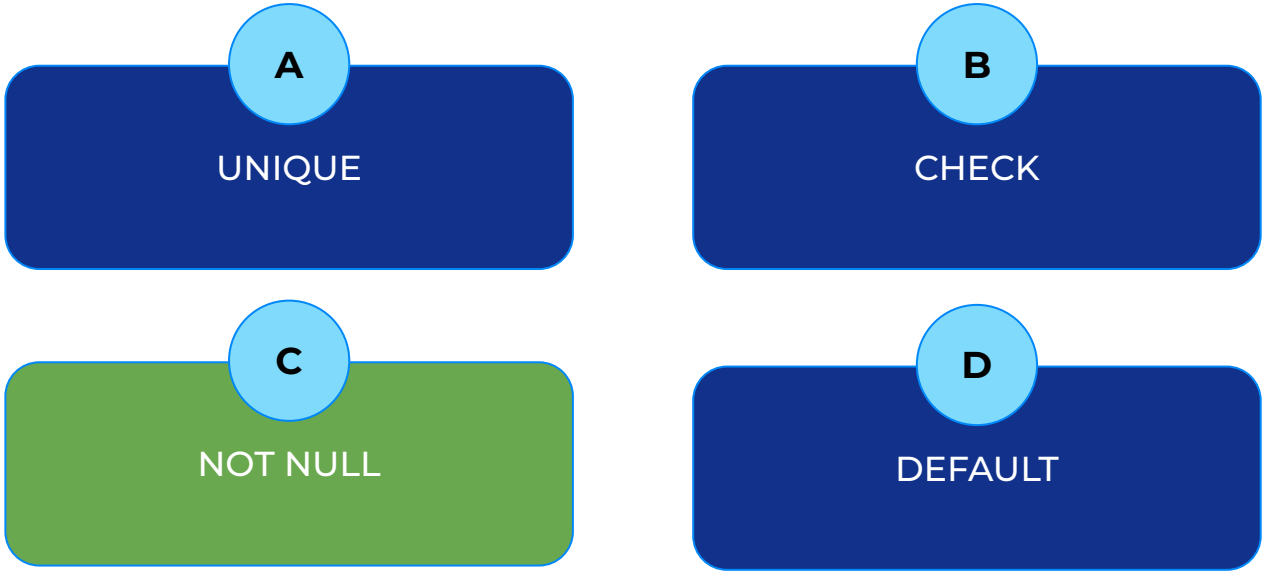
Question: Which constraint ensures a column cannot have NULL values?

A. UNIQUE

B. CHECK

C. NOT NULL

D. DEFAULT



Explanation:
The NOT NULL constraint enforces that a column must always have a value. Example:

CREATE TABLE employees (id INT NOT NULL);

- UNIQUE ensures distinct values, DEFAULT sets a fallback value, and CHECK validates data.

Thanks !

Thank You for attending the Session!!